

Guia Completo — API de Cadastro e Login com Node.js, TypeScript, Koa, JWT e MySQL

Este material foi criado como um guia extremamente detalhado para ensinar, passo a passo, como construir uma API profissional utilizando Node.js, TypeScript, Koa, MySQL e autenticação JWT. O objetivo é ensinar desde os conceitos mais básicos até a estrutura profissional de um backend moderno. Você aprenderá: - O que cada biblioteca faz - Como estruturar o projeto - Como funciona autenticação JWT - Como conectar no MySQL - Como proteger rotas - Como usar Models com Objection.js - Como separar responsabilidades corretamente - Como funciona cada linha de código Este guia foi escrito pensando em iniciantes e também em desenvolvedores que querem criar projetos mais profissionais.

1. O que vamos construir

Vamos criar uma API backend com as seguintes rotas:

POST /register

→ Responsável por cadastrar usuários

POST /login

→ Responsável por autenticar usuários

GET /me

→ Rota protegida que exige token JWT

Fluxo da aplicação:

1. O usuário se registra
2. A senha é criptografada com bcrypt
3. Os dados são salvos no MySQL
4. O usuário faz login
5. A API gera um token JWT
6. O token é usado para acessar rotas protegidas

2. Tecnologias utilizadas

Node.js:

Ambiente de execução JavaScript no servidor.

TypeScript:

Adiciona tipagem ao JavaScript.

Koa:

Framework minimalista para criação de APIs.

JWT:

Sistema de autenticação baseado em tokens.

bcrypt:

Biblioteca usada para criptografar senhas.

MySQL:
Banco de dados relacional.

Knex:
Responsável pela conexão e queries SQL.

Objection.js:
ORM usado para criar Models.

3. Criando o projeto

Comandos:

```
mkdir koa-auth-api  
cd koa-auth-api  
npm init -y
```

Explicação:

mkdir:
Cria uma pasta.

cd:
Entra dentro da pasta.

npm init -y:
Cria automaticamente o package.json.

O package.json funciona como o coração do projeto Node.js.

Ele armazena:

- dependências
- scripts
- nome do projeto
- versão
- configurações

4. Instalando dependências

```
npm install koa koa-router koa-bodyparser jsonwebtoken bcrypt mysql2 knex objection dotenv
```

Explicação detalhada:

koa:
Framework da API.

koa-router:
Sistema de rotas.

koa-bodyparser:
Converte JSON recebido em ctx.request.body.

jsonwebtoken:
Gera e valida tokens JWT.

bcrypt:
Criptografa senhas.

mysql2:
Driver de conexão com MySQL.

knex:
Query builder.

objection:
ORM baseado em Models.

dotenv:
Lê variáveis do arquivo .env.

5. Instalando dependências de desenvolvimento

```
npm install -D typescript ts-node-dev @types/node @types/koa @types/koa-router @types/jsonwebtoken @types/bcrypt @types/koa-bodyparser
```

Explicação:

-D:
Instala como dependência de desenvolvimento.

typescript:
Compilador TypeScript.

ts-node-dev:
Executa TypeScript automaticamente.

@types:
Tipagens TypeScript.

6. Estrutura profissional do projeto

```
src/  
■■■■ controllers/  
■■■■ services/  
■■■■ models/  
■■■■ routes/  
■■■■ middlewares/  
■■■■ database/  
■■■■ utils/  
■■■■ app.ts  
■■■■ server.ts
```

Controllers:

Recebem requisições HTTP.

Services:
Contêm regras de negócio.

Models:
Falam com o banco de dados.

Middlewares:
Executam lógica intermediária.

Routes:
Definem endpoints.

Utils:
Funções auxiliares.

7. Configurando TypeScript

Comando:

```
npx tsc --init
```

Arquivo tsconfig.json:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "esModuleInterop": true
  }
}
```

Explicação:

target:
Versão JavaScript gerada.

module:
Sistema de módulos.

rootDir:
Pasta do código fonte.

outDir:
Pasta final compilada.

strict:
Ativa verificações rígidas.

esModuleInterop:

Melhora imports.

8. Criando o arquivo .env

```
PORT=3000
```

```
DB_HOST=localhost  
DB_USER=root  
DB_PASSWORD=123456  
DB_NAME=koa_auth
```

```
JWT_SECRET=minha_chave_super_secreta
```

Explicação:

Variáveis de ambiente evitam deixar informações sensíveis diretamente no código.

9. Criando o banco de dados

```
CREATE DATABASE koa_auth;
```

```
USE koa_auth;
```

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explicação:

AUTO_INCREMENT:
Incrementa automaticamente.

PRIMARY KEY:
Identificador único.

UNIQUE:
Impede emails duplicados.

TIMESTAMP:
Data de criação.

10. Configurando Knex e Objection

Arquivo: src/database/knex.ts

```
import knex from 'knex'
import { Model } from 'objection'

const knexInstance = knex({
  client: 'mysql2',

  connection: {
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
  },
})

Model.knex(knexInstance)
```

```
export default knexInstance
```

Explicação linha por linha:

import knex:
Importa o Knex.

import { Model }:
Importa a classe base do Objection.

client:
Define o driver MySQL.

connection:
Configuração do banco.

Model.knex():
Liga o Objection ao Knex.

11. Criando o Model User

Arquivo: src/models/User.ts

```
import { Model } from 'objection'

export class User extends Model {
  id!: number
  name!: string
  email!: string
  password!: string

  static tableName = 'users'
}
```

Explicação:

extends Model:

Transforma a classe em um Model ORM.

static tableName:
Tabela usada no banco.

id!:
O símbolo ! informa ao TypeScript que o valor existirá.

12. Criando o service de autenticação

O service concentra regras de negócio.

Ele:

- valida usuário
- verifica senha
- cria hash
- acessa Models

Isso mantém controllers limpos.

13. Criptografia com bcrypt

bcrypt.hash(password, 10)

Explicação:

password:
Senha original.

10:
Quantidade de rounds.

Quanto maior o valor:

- mais seguro
- mais lento

bcrypt.compare():
Compara senha digitada com hash salvo.

14. JWT explicado detalhadamente

JWT significa JSON Web Token.

O token possui:

- Header
- Payload
- Signature

Quando o usuário faz login:

1. O backend valida as credenciais
2. Um token é criado
3. O frontend salva o token
4. O token é enviado no Authorization Bearer

Exemplo:

Authorization: Bearer TOKEN_AQUI

15. Middleware de autenticação

Middlewares executam antes da rota.

Nosso middleware:

- pega o token
- valida o token
- bloqueia acessos inválidos
- injeta dados do usuário no ctx

16. Explicando o Context do Koa

ctx representa a requisição inteira.

Exemplos:

ctx.body
→ resposta

ctx.request.body
→ body enviado

ctx.headers
→ headers HTTP

ctx.status
→ status da resposta

17. Explicando rotas

router.post('/login', login)

router:
Sistema de rotas.

post:
Método HTTP.

'/login':
Endpoint.

login:
Função executada.

18. Fluxo completo da aplicação

Cadastro:
Frontend → API → bcrypt → banco

Login:
Frontend → API → JWT → token

Rota protegida:
Frontend → token → middleware → acesso autorizado

19. Melhorias futuras

Você pode evoluir para:

- Refresh tokens
- Swagger
- Docker
- Prisma
- Testes automatizados
- Validação com Zod
- Clean Architecture
- DDD
- Rate limiting
- Logs
- CI/CD

20. Conclusão

Você construiu uma API profissional utilizando tecnologias modernas do mercado.

Os conceitos aprendidos aqui são utilizados em:

- fintechs
- bancos
- startups
- sistemas enterprise
- APIs REST modernas

Esse projeto é excelente para:

- portfólio
- entrevistas
- estudo

- evolução profissional